

App Deck - Technische Dokumentation

Version 1.9 vom 05.06.2026

1. Überblick

App Deck ist eine in Java Swing entwickelte Desktop-Anwendung für macOS, die als frei konfigurierbare Schaltflächen-Leiste (à la Stream Deck) fungiert. Die Anwendung erlaubt das Starten von URLs, Programmen, Dateien und Ordnern sowie das Kopieren von Text in die Zwischenablage. Die Schaltflächen sind auf mehreren Seiten und in Ordnern organisiert, können per Drag & Drop verschoben und in Echtzeit überwacht werden (YouTube-Update-Prüfung, laufende App-Erkennung).

2. Features

2.1 Schaltflächen-Raster

- 6 Spalten x 4 Zeilen = 24 Schaltflächen pro Seite
- Quadratische Buttons (120 x 120 Pixel) mit abgerundeten Ecken (ARC = 14)
- Hover-Effekt: hellblauer Verlauf (230,245,255 -> 200,225,245)
- Gedrückter Zustand: dunklerer Verlauf (210,210,215 -> 185,185,190)
- Schriftart: fett, 12pt
- Gold/orangener Fokus-Ring (4px, gestrichelt 8/6) bei Tastatur-Navigation

2.2 Schaltflächen-Typen

Typ	Beschreibung
URL	Öffnet eine Webseite im Standard-Browser, inkl. Favicon-Ladung
PROGRAMM	Startet ein Programm (macOS <code>open -a</code> oder direkter Pfad)
FILE	Öffnet eine Datei oder einen Ordner mit der Standard-Anwendung
FOLDER	Erzeugt eine Unterseite mit eigenem Raster und Navigation
COPY	Kopiert einen beliebigen Text in die Zwischenablage

2.3 Seiten und Navigation

- Mehrere Seiten über Pfeil-Buttons navigierbar
- Linke untere Ecke: Ruckwärts-Button (Pfeil, 28pt) - sichtbar wenn Seite > 0 oder in Ordner
- Rechte untere Ecke: Vorwärts-Button (Pfeil, 28pt) - immer sichtbar

- ESC (kurz, <800ms): Seite zurück (prevPage), dann Ordner verlassen (leaveFolder) - in dieser Reihenfolge
- ESC (lang, ≥800ms): Zur ersten Seite springen und Ordner verlassen (per System.currentTimeMillis()-Messung, kein Timer)
- Ordernavigation: Eintauchen in Unter-Ordner mit eigener Seiten-Struktur

2.4 Drag & Drop

- Schaltflächen können per Drag & Drop neu angeordnet werden
- Drag-Schwelle: 5 Pixel (reagiert schnell)
- Ghost-Fenster beim Ziehen: halbtransparente Vorschau
- Drop auf Pfeil-links: Verschieben auf vorherige Seite (mit Voll-Prüfung)
- Drop auf Pfeil-rechts: Verschieben auf nächste Seite (mit Voll-Prüfung)
- Drop auf Ordner: Verschieben in den Ordner (erste Seite)
- Drop auf andere Schaltfläche: Tausch der Positionen
- Hand-Cursor während des Drags

2.5 Laufende Apps erkennen

- Polling alle 5 Sekunden via `osascript` und `ps -ef`
- Grüner Rahmen (4px) um aktive Programm-Buttons
- 12-Sekunden-Cooldown nach manuellem Beenden
- Cooldown verfällt automatisch wenn App nicht mehr läuft

2.6 Langdruck zum Beenden

- Langdruck (>800ms) auf Programm-Button: App wird via `osascript` beendet
- Grüner Rahmen wird sofort entfernt
- 12-Sekunden-Sperre verhindert erneutes Erkennen der App
- Kein Dialog, keine Bestätigung

2.7 YouTube-Update-Prüfung

- Periodische Suche nach neuen Videos (Intervall: 5 Minuten)
- Erste Prüfung 10 Sekunden nach Start
- Extrahiert `videoId` aus YouTube-HTML via Regex
- Extrahiert `videoId` aus YouTube-ytInitialData-JSON via Gson (rekursive `videoRenderer`-Extraktion), Fallback: Regex auf HTML (erste 15 unique IDs)
- Alle gesehenen Video-IDs werden in `knownVideoIds` (List) gespeichert
- "Neu"-Badge (gold/orange) auf Schaltflächen mit neuen Videos
- Badge zeigt kumulierte Anzahl: "neu: X" (wird pro Prüfungsvorgang erhöht)
- Badge verfällt beim Klicken der Schaltfläche (setzt `newCount` auf 0)

- Ordner aggregieren “neu”-Anzahl aller Kind-Buttons
- Manuelle Prüfung über Menüpunkt (Cmd+Y)
- YouTube-Erkennung: Checkbox aktiviert sich automatisch bei YouTube-URLs
- Channel-Namen aus @handle extrahiert, og:title aus HTML geladen
- isYouTubeChannelUrl() filtert: nur URLs mit Pfad /@, /channel/, /c/, /user/ werden geprüft
- @-Handle wird automatisch via YouTube-Suche in Channel-ID aufgelöst (resolveYouTubeHandle())
- URL-Ausführung mit Runtime.exec("/usr/bin/open") statt Desktop.browse() um %40-Encoding zu vermeiden

2.8 Icons

- **URL-Buttons:** Favicons von Google s2/favicons (asynchron)
- **Programm-Buttons:** Icons aus .app-Bundle extrahiert via sips (ICNS -> PNG)
- **Datei-Buttons:** System-Icons via ShellFolder oder FileSystemView
- favicon.svg aus dem Verzeichnis wird unterstützt
- Programmgesteuerte Icons: Globus (URL ohne Icon), Ordner (gelb/orange), Kopieren (blaue Zwischenablage)
- Icon-Cache: ConcurrentHashMap (teilt sich zwischen EDT und Hintergrund-Threads)
- Cache wird bei Änderungen geleert (saveAndRefresh)

2.9 Suchdialog

- Öffnen via Cmd+F oder Type-to-Search (druckbare Zeichen bei nicht-fokussierten Textfeldern)
- Durchsucht alle Seiten (Root + Ordner) nach Label und Ziel
- Ergebnisformat: “Label (S.Seite > Ordnername > S.Subseite/Slot)”
- Navigation mit Pfeiltasten + Enter
- Doppelklick zum Auswählen
- Blinkender gelber Rahmen (5 Sekunden) um gefundenen Button
- Navigation zur richtigen Ordner-Unterseite

2.10 Tastatur-Navigation

- Pfeiltasten: Navigation zwischen den Buttons
- Enter: Aktion auf fokussiertem Button ausführen
- Cmd+F: Suchdialog öffnen
- Cmd+Y: YouTube-Prüfung manuell starten
- Cmd+B: Konfiguration sichern (Backup)
- Cmd+Shift+F: Fokus-Modus umschalten
- Cmd+Q: Anwendung beenden
- Cmd+D: Bedienungsanleitung anzeigen

- ESC (kurz): Seite zurück, dann Ordner verlassen
- ESC (lang $\geq 800\text{ms}$): Erste Seite, Ordner verlassen

2.11 Kontextmenu (Rechtsklick)

- **Bearbeiten...**: Öffnet den Konfigurationsdialog
- **Ordner anlegen**: Erzeugt einen neuen Ordner-Button
- **Entfernen**: Löscht die Schaltfläche
- **Als neu markieren**: Setzt das "neu"-Badge manuell

2.12 Konfigurationsdialog

- Label (Textfeld)
- Typ (Auswahl: URL, PROGRAM, FILE, FOLDER, COPY)
- Ziel (Textbereich mit 3 Zeilen, bei COPY 5 Zeilen mit Zeilenumbruch)
- Browse-Button für Dateiauswahl (mit Type-Ahead-Suche)
- Checkbox "Auf neue Videos prüfen" (nur bei YouTube-URLs)
- Auto-Detect: Ziel wird analysiert, Typ und Label automatisch vorgeschlagen
- Dialog: GridBagLayout, 15px Aussenrand, Buttons zentriert (OK/Abbrechen)
- Minimale Breite 520px, zentriert auf dem Bildschirm

2.13 Fokus-Modus

- Cmd+Shift+F: Dunkler Hintergrund füllt den gesamten Bildschirm
- Klick auf den Hintergrund leitet Fokus zurück an App Deck
- Menü-Aktionen (Cmd+F etc.) leiten Fokus zurück
- App Deck bleibt im Vordergrund bedienbar
- ESC beendet NICHT den Fokus-Modus (ESC ist reine Rückwärtsnavigation)

2.14 Logging

- Logdatei: `appdeck.log` im Konfigurations-Verzeichnis
- Format: `yyyy.MM.dd_HH:mm:ss` Nachricht
- Rotation bei 5 MB (9 Dateien: `appdeck.log` bis `appdeck.9.log`)
- Log-Level: Start, neue Videos mit Kanalname, Zusammenfassung, nächste Prüfung
- Keine HTTP-Status-, Fetch-Fehler- oder "keine Änderung"-Logs

3. Architektur

3.1 Projektstruktur

```
src/main/java/streamdeck/
  StreamDeckApp.java  - Hauptklasse (GUI, Logik, ~2589 Zeilen, Stand V1.9)
  ButtonConfig.java   - Datenmodell für eine Schaltfläche
  ConfigLoader.java   - JSON-Persistenz (Gson)

res/
  app-icon.png        - Anwendungs-Icon
  app-icon.icns       - macOS Icon (für .app-Bundle)

icons/
  app-icon.icns       - macOS Icon (für jpackage)

pom.xml               - Maven-Build (Gson 2.11.0, Shade-Plugin)

build-app.sh          - Build-Script für .app-Bundle (jpackage)
start-app-desk.sh     - Start-Script mit PID-basierter Single-Instance
raycaststartappdeskscrip.sh - Raycast-Integration
```

3.2 Datenmodell (ButtonConfig)

```
class ButtonConfig {
    String label;           // Anzeigename
    String type;            // URL, PROGRAM, FILE, FOLDER, COPY
    String target;          // Ziel-URL, Pfad, Befehl, Text
    List<List<ButtonConfig>> pages; // Nur bei FOLDER: Unterseiten
    boolean check;          // YouTube-Prüfung aktiv?
    List<String> knownVideoIds; // Alle bisher gesehenen Video-IDs
    int newCount;           // Kumulierter "Neu"-Zähler
}
```

3.3 ConfigData-Wrapper

```
class ConfigData {
    boolean focusMode;      // Fokus-Modus aktiv?
    List<List<List<ButtonConfig>>> pages; // Array-of-Array-of-Array (Seiten)
}
```

Die Konfiguration wird im JSON-Format als Objekt mit `focusMode` und `pages` gespeichert. Alte Array-of-Arrays-Formate (ohne Wrapper) werden automatisch erkannt und migriert.

Ein `configDirty`-Flag (`volatile boolean`) wird bei jeder Mutation gesetzt und im Shutdown-Hook abgefragt: Nur wenn `configDirty == true` wird

saveConfig() aufgerufen. Dies vermeidet unnötige Schreibzugriffe beim Beenden, wenn keine Änderungen vorgenommen wurden.

3.4 Konfigurationsformat (JSON)

```
[
  [ /* Seite 0 */
    { "label": "GitHub", "type": "URL", "target": "https://github.com",
      "check": false, "knownVideoIds": [], "newCount": 0 },
    { "label": "Terminal", "type": "PROGRAM", "target": "open -a Terminal" },
    { "label": "Dokumente", "type": "FOLDER", "target": "",
      "pages": [ [ { "label": "Bericht", "type": "FILE", "target": "/path/to/file.pdf" } ] ],
    { "label": "Passwort", "type": "COPY", "target": "meinPasswort123" }
  ],
  [ /* Seite 1 */
    ...
  ]
]
```

Mehrseiten-Format: Array-of-Arrays. Altes flaches Format wird automatisch erkannt und migriert (13er-Gruppen).

3.5 Konfigurations-Pfad (Fallback)

1. Aktuelles Arbeitsverzeichnis (config.json)
2. Neben dem .app-Bundle (Elternverzeichnis)
3. ~/Library/Application Support/App Deck/config.json

Bei Erststart: Dialog zur Auswahl einer vorhandenen oder Erstellung einer leeren Konfiguration.

3.6 GUI-Aufbau

```
JFrame (App Deck)
  JMenuBar
  JPanel ("bg", darkGradient = 35,35,40 -> 50,50,55)
    GridBagLayout (center)
      JPanel ("gridPanel", 6 Spalten x 4 Zeilen, FlowLayout/CENTER, 6px Abstand)
        JButton[24] (jeweils 120x120, Gradient 248,248,250 -> 225,225,230)
      BorderLayout.SOUTH
        JLabel (Version "V1.9 vom 05.06.26" in grau)
```

bg-Panel übersetzt mit GridBagLayout (anchor=CENTER, fill=NONE), dadurch bleibt gridPanel immer zentriert.

3.7 Buttons-Indizes

- `BOTTOM_ROW_START = (ROWS - 1) * COLS = 18`
- Letzter Slot (Index 23) = immer Vorwärts-Button
- Vorletzter Slot (Index 18) = Rückwärts-Button wenn Seite > 0 oder in Ordner
- `gridToPageIndex(int)`: Konvertiert Raster-Index zu Seiten-Index (berücksichtigt Navigations-Buttons)
- `pageToGridIndex(int)`: Umgekehrte Konvertierung

3.8 Fokus-Modus

- Ein separates `JWindow` (`fullscreen`, `unowned`, `setFocusableWindowState(false)`)
- `MouseListener` auf dunklem Panel ruft `toFront()` bei Klick
- `AWTEventListener` reagiert auf `ActionEvent` von `JMenuItem` und ruft `toFront()` mit 50ms Verzögerung
- `WindowFocusListener` als Sicherheitsnetz

3.9 Icon-Ladung

- Icons werden asynchron in einem Hintergrund-Thread geladen
- `iconCache` (`ConcurrentHashMap`) verhindert mehrfaches Laden
- Favicons: <https://www.google.com/s2/favicons?domain=DOMAIN&sz=64>
- macOS .app-Icons: `sips -s format png ICNS --out PNG`
- `favicon.svg`: `sips --resampleWidth WIDTH SVG --out PNG`
- `ShellFolder` via `Reflection` (Fallback: `FileSystemView.getSystemIcon`)
- Fallback beim Fehlschlagen: programmierter Globus

3.10 Laufende Apps erkennen

- `osascript` (Apple Events): Liefert alle Prozess-Namen
- `ps -ef`: Fallback für Shell-Befehle
- Polling alle 5 Sekunden via `javax.swing.Timer`
- Extraktion des App-Namens aus dem Ziel-String (verschiedene Formate: `open -a NAME`, `open "PFAD"`, `.app-Datei`)
- Vergleich: `String.toLowerCase().contains(appKey)`

3.11 YouTube-Check

- `ScheduledExecutorService` mit `scheduleWithFixedDelay`
- Erstverzögerung: 10s, Intervall: 5 Minuten
- HTTP-GET auf YouTube-URL, Parsen von `ytInitialData` JSON via `Gson` (rekursive `videoRenderer`-Extraktion)
- Fallback: Regex nach `"videoId": "..."` (erste 15 unique IDs)
- Vergleich mit gespeicherter `knownVideoIds`-Liste
- Bei neuen Videos: `newCount += Differenz`, Log-Eintrag mit Kanalname

- `knownVideoIds` wird nach jedem Check aktualisiert (akkumulierend)
 - `Badge (newCount)` wird beim Klicken der Schaltfläche auf 0 gesetzt
 - `isYouTubeChannelUrl()` filtert: nur Kanal-URLs werden geprüft (`/@`, `/channel/`, `/c/`, `/user/`)
 - `@-Handle` wird automatisch in Channel-ID aufgelöst via `resolveYouTubeHandle()` (YouTube-Suche)
-

3.12 configDirty-Flag

```
volatile boolean configDirty = false;
```

Wird bei jeder Änderung auf `true` gesetzt: - Ordner anlegen / Entfernen / Als neu markieren - Drag & Drop (alle Varianten) - Edit-Dialog (OK) - YouTube-Check bei neuen Videos - Button-Klick setzt `newCount` zurück

Klarstellung in `saveConfig()` -> `configDirty = false`. Shutdown-Hook: `if (configDirty) saveConfig();`

4. Voraussetzungen

- **Java 21+** (JDK 21+ für Build und Ausführung)
 - **macOS** (für native Funktionen: `osascript`, `sips`, `open`, `jpackage`)
 - **Maven** (für Build)
 - **Pandoc + XeLaTeX** (für PDF-Dokumentation)
 - **IntelliJ IDEA** (empfohlen für Entwicklung)
-

5. Installation

5.1 Aus dem Quellcode bauen

```
git clone git@github.com:martindobronski/app-deck.git
cd app-deck
./build-app.sh
```

Das Script führt `mvn package` und `jpackage` aus und erzeugt `App Deck.app`.

5.2 Manuell starten

```
mvn package -q
java -jar target/streamdeck-1.9.jar [config.json]
```


5.3 .app-Bundle starten

open "App Deck.app"

5.4 Single-Instance-Script

./start-app-desk.sh

Startet nur eine Instanz (PID-Datei in /tmp/app-desk.pid).

6. Bedienung

6.1 Schaltflächen konfigurieren

Rechtsklick -> "Bearbeiten..." oder auf leere Schaltfläche klicken. Der Konfigurationsdialog erlaubt:

- Label: Anzeigename
- Typ: URL, PROGRAM, FILE, FOLDER, COPY
- Ziel: URL, Pfad, Befehl, Text
- "Auf neue Videos prüfen": Nur für YouTube-URLs

Der Typ und Label werden automatisch erkannt:

- http://... -> URL (Label aus Domain)
- open -a ... -> PROGRAM (Label aus App-Name)
-app -> PROGRAM (Label aus Dateiname)
- file://... -> FILE (Label aus Pfad)

6.2 Schaltflächen anordnen

Drag & Drop: Button an die gewünschte Position ziehen.

- Auf Pfeil-links/rechts: Seite wechseln
- Auf Ordner: in Ordner verschieben

6.3 Programme beenden

Button länger als 800ms gedrückt halten. Die App wird via `osascript` beendet.

6.4 Suchen

Cmd+F oder einfach tippen: Durchsucht alle Seiten und Ordner nach Label/Ziel.

6.5 Konfiguration sichern

Menupunkte:

- App Desk -> Konfiguration sichern (Cmd+B): Kopiert config.json nach bak/YYYYMMDD_HHmss_config.json
-

7. Technische Details

7.1 Build (build-app.sh)

```
ARTIFACT_VERSION=$(grep -m1 '<version>' pom.xml | sed 's/.*<version>\(.*\)</version>.*\/1/  
mvn package -q                                     # Erzeugt target/streamdeck-$ARTIFACT_VERSION.jar  
rm -rf dist/                                         # Bereinigen  
jpackage \                                           # .app-Bundle erzeugen  
  --type app-image \  
  --name "App Deck" \  
  --app-version "$ARTIFACT_VERSION" \  
  --icon icons/app-icon.icns \  
  --input target/ \  
  --main-jar "streamdeck-$ARTIFACT_VERSION.jar" \  
  --main-class streamdeck.StreamDeckApp \  
  --mac-package-identifier com.dobronski.appdeck \  
  --dest dist/
```

7.2 Abhängigkeiten

Abhängigkeit	Version	Verwendung
Gson	2.11.0	JSON-Persistenz
Maven Shade	3.6.0	Fat-JAR (alle Dependencies)

7.3 macOS-spezifische Funktionen

- osascript: Apple Events für App-Liste und App-Beenden
- sips: Konvertiert ICNS -> PNG und SVG -> PNG
- open: Startet .app-Bundles und öffnet URLs/Dateien
- jpackage: Erzeugt das .app-Bundle
- ShellFolder: Holt System-Icons (via Reflection)

7.4 Thread-Sicherheit

- Icon-Cache: ConcurrentHashMap (EDT und Hintergrund-Thread)
- YouTube-Check: ScheduledExecutorService (einzelner Daemon-Thread)

- GUI-Updates: `SwingUtilities.invokeLater`
- Drag & Drop: EDT (`MouseListener`)
- Running Apps: EDT (`javax.swing.Timer`)

7.5 Farben und Design

Element	Farbe(n)
Hintergrund	Verlauf 35,35,40 -> 50,50,55
Button normal	Verlauf 248,248,250 -> 225,225,230
Button Hover	Verlauf 230,245,255 -> 200,225,245
Button gedrückt	Verlauf 210,210,215 -> 185,185,190
Rahmen (running)	Grün (0,180,0), 4px
Rahmen (normal)	Grau (170,170,175), 1px
Fokus-Ring	Gold/orange (255,200,0), 4px, gestrichelt 8/6
Badge	Gold (255,200,0), Schrift (255,255,255) auf (200,140,0)
Version-Label	Grau

7.6 Border-Implementierung

`RoundedShadowBorder` (Custom `AbstractBorder`):

- Abgerundete Ecken (ARC = 14)
- Schatten (3px, schwarz mit alpha=35)
- Konfigurierbare Linienfarbe und -stärke
- `getBorderInsets`: 4,4,7,7 (plus Schatten)

8. Bekannte Einschränkungen

- macOS-only: `osascript`, `sips`, `open`, `jpackage`, `ShellFolder`
- Kein nativer Windows-/Linux-Support (in Planung als Single Code-Base)
- YouTube-Check kann fehlschlagen wenn YouTube Consent-Redirects ausliefert
- Favicons nur von Google `s2/favicons` (keine direkte Seitenextraktion)
- `ShellFolder` nutzt Reflection (kann auf JDK 9+ scheitern)

9. Lizenz

Proprietär. Alle Rechte vorbehalten.